System Calls

Lecture Notes — Week 3

Mechanics, Parameter Passing, and Types

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 17, 2026

Concepts That Click

3.1 The Command That Keeps Knocking

Run the command

```
cp notes.txt backup.txt
```

and four things happen in order: the program opens one file, creates another, copies every byte from the first into the second, and closes both. The command looks like a single action at the shell prompt, but it is not a single action inside the machine. The question that organizes this chapter is the one the command keeps raising: *how many times does the program cross from user mode into the kernel, and what does it say each time?*

The answer cannot be found by reading the source code of `cp` alone. `cp` is ordinary user code. It cannot touch the disk, modify the page tables, or read another process's memory, because the hardware forbids those operations in user mode. Every privileged operation it needs must be requested through a **system call**. Each request has four parts: a name, a number, a set of arguments, and a return value. The rest of the chapter is about those four parts — how they are expressed, how they cross the boundary, and how the kernel decides whether to honour them.

We will keep `cp notes.txt backup.txt` as the running thread. Every mechanism we introduce will be tied back to a step in that trace.

Roadmap

Four questions frame the chapter, and they are also the four questions a programmer must be able to answer about any operating system:

1. **What is a system call?** It is the kernel's programming interface — the boundary between a program and the services the OS provides.
2. **How does control cross the boundary?** Through the trap and the system-call table.
3. **How do arguments travel?** Through registers, a memory block, or the stack.
4. **What kinds of calls exist?** The six standard categories: process control, file management, device management, information maintenance, communication, and protection.

The division of labour is worth stating once. The *mechanism* that carries a request across the boundary is hardware; the *interface* the programmer uses is software; and the *taxonomy* of calls is the vocabulary the rest of the course will use.

3.2 Two Views of the Same Service

Reading: Silberschatz et al., Ch. 2 §2.3, pp. 55–58.

Programmers rarely invoke a system call directly. They call a library function, and the library eventually executes the trap. This is not an accident of history; it is the normal layering of a C program, and it produces two views of one service. Table 3.1 sets the two views side by side.

Programmer's view (API)	**Kernel's view (system call)**
`fopen`, `fread`, `fwrite`, `fclose`	`open`, `read`, `write`, `close`
Standard C library; buffered; portable	Kernel entry points; unbuffered; architecture-specific
Returns a `FILE *` or a count	Returns a file descriptor or -1 with `errno`

Table 3.1: Two views of the same file service. The API is what the programmer writes; the system call is what the kernel executes.

The left column is a *convenience layer*. It knows about buffering, it hides the details of the machine's instruction set, and it offers a uniform interface across operating systems. The right column is the *actual request* to the kernel: an entry point with a fixed number, a fixed calling convention, and a return value the kernel can produce.

The two columns are not in one-to-one correspondence. One library call may issue zero, one, or many system calls, because buffering changes the count. A call that reads from a buffer the library already filled may never trap at all; a call that fills a buffer may trap once for a large transfer; a call that flushes may trap for each underlying write. This is why counting the system calls in a program requires looking at the library's behaviour, not just at the number of function calls in the source. Silberschatz et al. describe this layering in Ch. 2 §2.3 (pp. 55–58) [1].

3.3 What Exactly Is a System Call?

Definition (System call). *A* ***system call*** *is the programmatic request from a user program to a kernel service. It is the boundary of the operating system's programming interface.*

The definition sounds like a function call, and at the source level it looks like one: a name, parentheses, arguments, and a return value. But it is *not* an ordinary function call. Three differences matter.

First, it *changes the privilege level*. The CPU leaves user mode, enters kernel mode, runs kernel code, and returns to user mode. An ordinary function call never changes the privilege level; caller and callee execute with the same rights.

Second, the kernel executes the request *on behalf of the calling process* and *with the kernel's privileges*. The process is still the owner of the request — the open file will belong to it, the bytes read will land in its buffer — but the code that performs the operation is trusted kernel code.

Third, the kernel *validates the request before acting on it*. The user program supplies arguments, but the kernel does not assume they are sensible. This third difference is the one that turns the boundary from a convenience into a security mechanism, and we will return to it repeatedly.

One warning about terminology: the same phrase "system call" is used for three different things — the API function the programmer calls, the trap instruction that crosses the boundary, and the kernel routine that ultimately runs. Context tells you which layer is meant. When this chapter says "the system call returns a file descriptor," it means the kernel routine; when it says "the system

Concepts That Click

call is invoked with `syscall`," it means the instruction. The distinction is not pedantry: conflating the layers is the source of most of the misconceptions at the end of the chapter.

3.4 Why Not Link the Kernel Into Every Program?

If a system call is just a request for a service, why not avoid the boundary altogether? Why not link the kernel's code into every program, the way a library is linked, and let each program call the routines directly? The answer is that four properties would be destroyed at once.

- **Protection.** User code would run with kernel privileges. A single buggy program could then corrupt the machine or overwrite the kernel itself.
- **Isolation.** One buggy program could overwrite another program's memory or the kernel's data structures, because nothing would prevent it from addressing them.
- **Portability.** The same API can sit on different kernels and different instruction sets only because the boundary hides the implementation. Linking the kernel in would bind every program to one kernel's internals.
- **Evolution.** The kernel can change its internal data structures and algorithms without recompiling every application, because the application sees only the stable interface, not the implementation.

The trade-off is stated just as plainly: every system call costs a mode switch. The design goal is therefore to keep the boundary *narrow* — few, well chosen entry points — and the calls *efficient*. The rest of the chapter is, in a sense, the engineering of that trade-off: how to cross the boundary with as little overhead as possible while still validating everything that crosses.

3.5 Why `printf` Is Not a System Call

A useful test of the API/system-call distinction is the most familiar C function of all. `printf` is not a system call. What does it eventually call, and why does the C library buffer the output first?

`printf` formats its arguments into a *user-space buffer*. The formatting is ordinary library code running in user mode; none of it needs the kernel. When the buffer fills, or when the program (or the library) decides to flush it, the library calls `write`, and `write` traps into the kernel. The kernel then copies the buffer to the destination.

Buffering exists to reduce the number of expensive mode switches. If every `printf` trapped immediately, a program printing a line in twenty small pieces would pay for twenty crossings. With a buffer, the same twenty pieces become one `write` and one crossing. The bytes delivered are the same; the number of boundary crossings is not. Table 3.1's warning that one library call may issue zero, one, or many system calls is exactly this effect.

Concepts That Click

3.6 The Trap Path

Reading: Bovet & Cesati, Ch. 10, pp. 398–419.

The trap instruction is the hinge of the whole mechanism: one instruction moves the CPU from user mode to kernel mode and hands the kernel a number. Figure 3.1 shows the path that instruction starts.

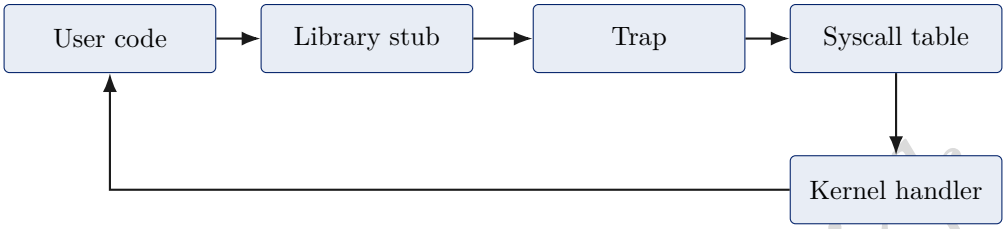


Figure 3.1: The trap path. User code calls a library stub; the stub executes the trap; the kernel indexes its table by number, runs the handler, and returns to the user program. The mechanism is described in Silberschatz et al., Ch. 2 §2.3, pp. 55–58 [1], and in concrete Linux detail in Bovet & Cesati, Ch. 10, pp. 398–419 [3].

The path has three stages. In the first stage, the library stub places the **system-call number** and the arguments in registers. The number is what identifies the requested service; the registers are where the kernel will look for the arguments. In the second stage, the trap instruction saves the user program counter (PC) and flags, switches the CPU to kernel mode, and jumps to the kernel's trap entry point. Saving the PC is what makes a return possible: the kernel must be able to resume the user program at the instruction after the trap. In the third stage, the kernel indexes the **system-call table** by the number it received and calls the corresponding handler. When the handler finishes, the kernel restores the user context and returns.

The arrows in Figure 3.1 are drawn as a path, not as a cycle, because the user program does not continue while the kernel works. Control has moved to the kernel; the user code resumes only after the handler returns.

3.7 Worked Example: `write(fd, buf, n)`

The general path becomes concrete when we follow a single call. Consider

```
write(fd, buf, n)
```

where `fd` is a file descriptor, `buf` is the address of a user buffer, and `n` is the number of bytes to write.

Example 3.1. The seven steps of a `write` system call.

1. *`fd`, `buf`, and `n` are placed in the argument registers. This is still user-mode code; it is the library stub arranging the arguments where the kernel's calling convention expects them.*
2. *The system-call number for `write` is placed in a designated register. The number and the arguments together describe the entire request: which service, and on what data.*
3. *The stub executes `syscall` (x86-64) or `int 0x80` (legacy x86). This is the instruction that actually crosses the boundary.*

Concepts That Click

4. *Hardware switches to kernel mode, saves the return address, and enters the trap handler. The return address is the user PC, saved so that the program can resume after the kernel finishes.*

5. *The kernel validates* `fd`*, checks that* `buf` *lies in user memory, and performs the write. Validation comes first: the kernel will not copy to or from an address it has not checked.*

6. *The return value is placed in a register; the kernel restores user mode and returns. The calling program continues at the saved return address.*

7. *On error, the library sets* `errno` *and returns* -1*. The kernel signalled the error through its return value; the library translates it into the C convention the programmer expects.*

Steps 1–2 run in user mode, steps 3–6 run in the kernel, and step 7 runs back in user mode. The boundary is crossed on entry (steps 3–4) and again on return (step 6); the seven steps therefore contain one complete round trip. Bovet and Cesati follow this path in detail for Linux in Ch. 10 (pp. 398–419) [3], and Silberschatz et al. give the architecture-neutral account in Ch. 2 §2.3 [1].

3.8 The System-Call Table

Definition (System-call table). *A kernel array indexed by system-call number. Each entry is a function pointer to the handler for that service.*

The table is the second half of the hinge. The trap instruction carries only a number; it does not carry a function name or an address chosen by the user program. The table turns that number into the right kernel routine, and it does so without a long search: the number is an index, so the kernel can go directly to the entry.

Four properties follow from this design.

- The number is the **ABI** (application binary interface): it must stay stable across kernel versions. A compiled program contains the number, not the name, so changing the number would break the program.
- Adding a call means adding a table entry and a handler — not changing the trap instruction. The instruction is generic; the meaning lives in the table.
- The handler runs in kernel mode with the kernel's privileges, which is why the validation in step 5 of the worked example is possible at all.
- The table is small and indexed, so the cost of dispatch is a lookup rather than a search through names.

On Linux x86-64, `write` is system call 1, and the table lives in the kernel's trap entry code. The number is not a historical accident to be memorized for its own sake; it is the concrete form of the contract between a compiled binary and the kernel it runs on.

3.9 Return Values and `errno`

A system call must report three kinds of outcome: success with a value, success with no data (end of file), and failure. The kernel reports them through a register. The value in that register is either a count, a file descriptor, or an error code.

Concepts That Click

The convention is that a *negative* return value signals an error. The library wrapper translates that negative value into `-1` and sets `errno` to a code identifying the failure. The caller must check the return value, because the kernel cannot print a message for it.

The same discipline governs the arguments. The kernel never trusts user pointers or lengths; it validates them before copying. A pointer that is syntactically valid — a number of the right width — may still point somewhere the process is not allowed to touch. Validation is the kernel's job, and it happens before any data moves.

Example 3.2. The three outcomes of `read`. *A `read` call returns the number of bytes read on success. It returns `0` at end of file, which is a successful call that transferred no bytes. It returns* -1 *on error, with `errno` set to a code such as `EFAULT` (a bad user address) or `EBADF` (a bad file descriptor), or a similar value. The three cases are distinguished by the return value, so a correct program must inspect it rather than assume the call succeeded.*

3.10 Three Ways to Pass Arguments

Reading: Silberschatz et al., Ch. 2 §2.3, pp. 55–58.

A system call is a function call across a privilege boundary. The arguments must cross too — and the kernel must not trust any of them. There are three standard mechanisms for getting the arguments across, and Figure 3.2 arranges them by their trade-off.

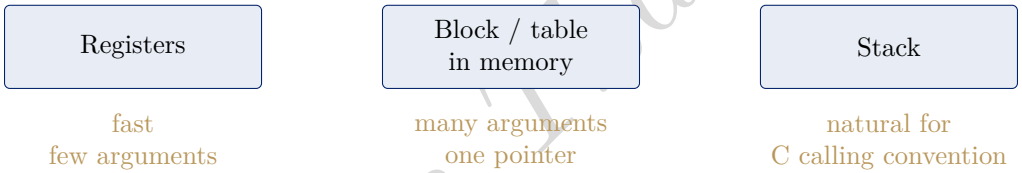


Figure 3.2: The three standard mechanisms for passing system-call arguments. Silberschatz et al. describe all three in Ch. 2 §2.3, pp. 55–58 [1].

Silberschatz describes all three; real ABIs choose one or a mix. The choice is not cosmetic, because the kernel must know where to find the arguments before it can validate them. A mechanism that is fast but limited, a mechanism that scales to many arguments but requires an extra indirection, and a mechanism that fits an existing calling convention are three different answers to the same engineering question.

3.11 Method 1: Registers

The caller places the first few arguments in designated registers. This is the fastest mechanism: the values are already in the CPU, so no memory access is needed beyond the registers that were already saved on entry. The limitation is equally clear: the method is bounded by the number of argument registers. A call with more arguments than the ABI reserves cannot use this method alone.

Example 3.3. Linux x86-64 `read(fd, buf, count)`. *The convention for this call is:*

`rax`	*system-call number for* `read`
`rdi`	`fd`
`rsi`	`buf` *(user virtual address)*
`rdx`	`count`

Concepts That Click

The number travels in `rax`; the three arguments travel in `rdi`, `rsi`, and `rdx`. The choice of registers is part of the ABI, so the library stub and the kernel agree on it without negotiation.

Register passing does not relax validation. The kernel still checks that `buf` is a legal user address; putting the address in `rsi` says where the buffer is, not that the program is entitled to use it.

3.12 Method 2: Block or Table in Memory

When the argument list is too large for the reserved registers, the caller can store the arguments in a memory block and pass the *address* of the block in a register. The kernel copies the block into kernel space and validates every field. One register now carries an arbitrary number of arguments, at the price of one indirection and a copy.

Example 3.4. ***A call with many arguments.*** *`open(path, flags, mode)` can be passed as a table holding the pointer to `path`, the value of `flags`, and the value of `mode`. A single register holds the address of the table; the kernel reads the table once, copies it into kernel memory, and validates the pointer to `path` along with the other fields.*

The pointer to the block is itself untrusted. The kernel must verify that the *whole* block lies in the caller's address space before it reads any field, because a block that starts in user memory and continues into kernel memory would be just as dangerous as a direct pointer into the kernel.

3.13 Method 3: Stack

The caller can also push the arguments onto the user stack, from which the kernel reads them using the saved user stack pointer. This is natural for languages and calling conventions that already pass arguments on the stack: the stub does not have to rearrange anything, because the arguments are already laid out in the expected place.

Whichever method is used, the kernel must satisfy the same three requirements:

1. *Locate* the arguments — know which registers, block, or stack slots hold them.
2. *Copy* them into kernel memory, so that the kernel is not reading a user buffer that the user program could change underneath it.
3. *Validate* every pointer and length before using it, so that a malformed argument produces an error rather than a corrupted kernel.

These three steps are the invariant behind the three mechanisms. The mechanisms differ in where the arguments start; they do not differ in what the kernel must do once it has found them.

3.14 Pointer Validation: Never Trust User Memory

A user pointer is just a number, and it may point anywhere: into kernel memory, into another process's memory, or into unmapped memory that will fault on access. The kernel therefore checks the range against the process's address space before it uses the pointer. It then copies data with special routines (`copy_from_user`/`copy_to_user` on Linux) that handle faults safely. A bad pointer returns `EFAULT`; it does not crash the kernel.

Concepts That Click

The security rule is absolute: every argument is untrusted input. Validation happens in kernel mode, before the kernel acts on the request. This is the reason the boundary exists. If the kernel trusted a user pointer, a program could pass the address of a kernel data structure to `read` and receive the kernel's memory as output, or pass it to `write` and overwrite kernel state. Dereferencing the pointer directly would make the protection of the previous section meaningless.

A concrete test makes the rule vivid. Suppose a user program passes a pointer to a kernel address to `read`. The kernel must not follow the pointer. It checks that the range lies in the user address space; if it does not, the call fails with `EFAULT`. The program learns that its request was invalid, and the kernel remains intact. The check is not an optimization; it is the boundary being enforced.

3.15 The Six Categories

Reading: Silberschatz et al., Ch. 2 §2.4, pp. 58–66.

System calls are not a random list. They fall into six categories, and every command you run is a combination of them. Table 3.2 gives the taxonomy.

Category	Purpose	Typical calls
Process control	create, terminate, wait, load	`fork`, `exec`, `wait`, `exit`
File management	create, open, read, write, close	`open`, `read`, `write`, `close`, `lseek`
Device management	request/release, read/write, control	`ioctl`, `read`, `write`
Information maintenance	get/set time, process, file attributes	`getpid`, `alarm`, `sleep`, `stat`
Communication	create/delete connections, send/receive	`pipe`, `mmap`, `socket`, `send`, `recv`
Protection	set/get permissions	`chmod`, `umask`, `chown`, `setuid`

Table 3.2: The six categories of system calls. Silberschatz et al., Ch. 2 §2.4, pp. 58–66 [1].

The categories are not a hierarchy and they are not mutually exclusive in practice: a single operation may touch several of them. They are a vocabulary for describing what a call is *for*, and the rest of the course will use that vocabulary constantly.

3.16 Process Control

The process-control calls manage the life cycle of a process.

- `fork`: create a child process that is a copy of the parent. The child inherits the parent's open-file table.
- `exec`: replace the current process image with a new program, so the process runs different code.
- `wait`: block until a child terminates and collect its status. This is how a parent learns that a child has finished and whether it succeeded.
- `exit`: terminate the calling process and release its resources.

Concepts That Click

The shell's job is the canonical combination. A shell `fork`s, the child `exec`s the command, and the parent `wait`s. When you type `cp notes.txt backup.txt`, the shell creates a child, the child replaces itself with the `cp` program through `exec`, and `cp` itself is the child after `exec`. The shell is still waiting; that is why the prompt returns only when the copy finishes.

3.17 File Management

The file-management calls operate on files and on the descriptors that represent them.

- `open`: create or open a file, returning a file descriptor.
- `read`/`write`: transfer bytes between user memory and the file.
- `lseek`: move the file offset without transferring data.
- `close`: release the descriptor and flush state.
- `stat`: read metadata (size, owner, permissions, times).

Definition (File descriptor). *A small integer index into the process's per-process open-file table — not the file itself.*

The distinction in the definition matters. The descriptor is a handle, and it is meaningful only in the process that owns it; the open-file table entry behind the handle is what actually refers to the file.

3.18 Device, Information, Communication, Protection

The remaining four categories cover everything that is not process or file management.

Device management. `ioctl` controls device-specific behaviour; `read` and `write` move data. Devices are often accessed through the same descriptor interface as files, which is why the same `read` and `write` calls appear in both the file-management and device-management rows of Table 3.2. The uniformity is a deliberate design choice: a program that knows how to read a file already knows how to read from many devices.

Information maintenance. `getpid`, `alarm`, `sleep`, and `time` retrieve or manipulate information about the process and the system; `stat` retrieves file metadata. These calls do not move bulk data; they query or set small pieces of state.

Communication. `pipe`, `mmap`, `socket`, `send`, and `recv` move data between processes or across a network.

Protection. `chmod`, `umask`, `chown`, and `setuid` set or query access rights. They do not read or write file contents; they change who is allowed to do so.

Concepts That Click

3.19 Worked Example: `cp` Across the Categories

The six categories stop being abstract when we lay the running thread out against them.

Example 3.5. `cp notes.txt backup.txt` as a sequence of system calls.

1. ***Process control:*** *the shell `fork`s and the child `exec`s `cp`.*
2. ***File management:*** *`open` is called on `notes.txt` with `O_RDONLY`, and on `backup.txt` with `O_WRONLY|O_CREAT`, which creates or truncates the destination.*
3. ***File management:*** *a loop of `read` into a buffer and `write` to the destination moves the bytes.*
4. ***File management:*** *`close` releases both descriptors.*
5. ***Process control:*** *`exit(0)` ends the child; the shell's `wait` collects the status.*

One shell command, ten or more system calls. The count is worth deriving explicitly, because it is the answer to the question the chapter opened with. There is one `fork`, one `exec`, two `open` calls, two `close` calls, one `exit`, and one `wait` — eight calls before any data moves. At least one `read` and one `write` are needed even for a one-byte file, which brings the minimum to ten. A real file needs many more iterations of the read/write loop, so the count grows with the file size. The categories are not abstract: they are the steps in the trace.

3.20 Why `fork` Is Process Control

A final classification check. Which category does `fork` belong to, and why is it not file management even though the child may open files?

`fork` creates a new process, so it is process control. The child inherits the parent's open-file table, but `fork` itself does not create or access a file. The category describes the *operation*, not the resources the process may use afterward. This is the rule to apply to every call in the taxonomy: classify the operation, not its consequences.

3.21 Numbers, ABI, and Compatibility

Reading: Bovet & Cesati, Ch. 10, pp. 398–419; Silberschatz et al., Ch. 2 §2.3, pp. 55–58.

The interface is stable, but every call has a cost. Good systems programmers batch calls and know when the boundary is being crossed. Before turning to the cost, it is worth seeing why the interface is stable in the first place.

Each architecture defines a system-call number for each service. The numbers are part of the **application binary interface**. A kernel cannot renumber existing calls without breaking every compiled program, because the number is embedded in the binary, not looked up by name at run time. New calls are therefore added at the end of the table, and old calls remain for compatibility.
Definition (Application binary interface). *The contract between compiled code and the kernel: the calling convention and, for system calls, the number assigned to each service.*

The consequence is that the system-call table is a long-lived contract between the kernel and every binary that runs on it. The contract can grow, but it cannot be silently rewritten.

Concepts That Click

3.22 The Cost of Crossing the Boundary

A system call is not free. The cost has four components:

- Save the user context and switch to kernel mode.
- Index the table and enter the handler.
- Validate the arguments and copy data across the boundary.
- Perform the operation, then restore the user context.

Each component is small, but they add up, and the copy in the third component is proportional to the amount of data moved. This is why batching matters. Copying 1 MB in one `read` costs one mode switch; copying it in 1-byte chunks costs roughly a million. The bytes are the same; the crossings are not. The same arithmetic applies to writes, and it is the reason library buffering (recall `printf`) is worth the complexity.

`strace` shows the actual calls a program makes. It is the practical way to see the boundary in action: instead of guessing how many times a program traps, you can read the sequence of calls it really issued.

3.23 Common Misconceptions

Table 3.3 collects the four errors that the chapter has been building the tools to correct.

Wrong	**Right**
"`printf` is a system call."	`printf` uses buffered `write` calls.
"A system call is just a function call."	It changes privilege level and uses a kernel entry point.
"The kernel can trust the user's pointer."	User pointers and lengths are untrusted and validated.
"Every library call traps into the kernel."	Many library calls stay entirely in user space.

Table 3.3: Four common misconceptions about system calls, and the correction for each.

Each correction is a one-line summary of a mechanism developed earlier: the first of the API/system-call distinction (§3.2 and §3.5), the second of the trap path (§3.6), the third of pointer validation (§3.14), and the fourth of the fact that one library call may issue zero, one, or many system calls (§3.2).

3.24 Summary

A system call is the program's request to the kernel; it is invoked through a trap, not an ordinary call. The system-call number selects a handler from the kernel's dispatch table, and the table is a long-lived ABI contract. Arguments travel in registers, in a memory block, or on the stack, and every pointer and length is validated before the kernel acts. The six categories — process control, file management, device management, information maintenance, communication, and protection — organize the interface, and a single command such as `cp notes.txt backup.txt` is a combination

Concepts That Click

of them. Mode switches are expensive, so work is batched to reduce them; the library API sits on top of the system calls and decides, through buffering, how many crossings a program actually pays for.

The chapter's opening question now has a concrete answer. `cp` crosses the boundary at least ten times for a one-byte file — once per system call, with a matching return crossing for each — and the number grows with the file size because the read/write loop repeats. What it says each time is the system-call number, followed by arguments the kernel validates before it acts.

3.25 Looking Ahead and Further Reading

This chapter covered how a program names a kernel service, how the arguments cross the boundary, and what kinds of calls exist. The next lecture moves up one level: it examines the user-level programs built on top of these calls, and the internal structures — monolithic, layered, microkernel, modular, and virtual machine — that organize the kernel itself. The reading for that lecture is Silberschatz et al., Ch. 2 §2.5–2.8 [1], and Stallings, Ch. 2 and Ch. 4 §4.3 [2].

For this chapter, the page-cited sources are:

- A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 8th ed. — Ch. 2 §2.3 (system calls, pp. 55–58) and §2.4 (types of system calls, pp. 58–66).
- D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. — Ch. 10 (system calls, pp. 398–419), for the concrete x86/Linux entry and exit path.
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6th ed. — Ch. 3 (process description and control), for the process context in which calls execute.

All three citations are page-verified through each book's `index.md` in the course's supporting-books directory:

`master_supporting_docs/CS502/supporting_books/`

Self-Check

Exercises

1. List the system calls in the `cp notes.txt backup.txt` trace, classify each by category, and state the minimum number of user-to-kernel crossings for a one-byte file. Explain why the count grows with the file size.
2. A program calls `printf` one hundred times, one byte per call, and then exits. Does that imply one hundred `write` system calls? Explain using the user-space buffer and the conditions under which a flush occurs.
3. A hypothetical system call takes seven arguments. Which of the three argument-passing mechanisms would you choose, and what must the kernel do before it uses the arguments? Be specific about the pointer.
4. A user program passes a pointer to a kernel address to `read`. What should happen, and why can the kernel not simply dereference the pointer?

Concepts That Click

5. A program copies a 4 MB file in 4 KB chunks, with one `read` and one `write` per chunk. Compare the number of system calls with a single 4 MB `read` followed by a single 4 MB `write`. Which cost dominates in each case?

Solutions

1. The calls are `fork` (process control), `exec` (process control), two `open` calls (file management), at least one `read` and one `write` (file management), two `close` calls (file management), `exit` (process control), and the shell's `wait` (process control). For a one-byte file that is $1 + 1 + 2 + 1 + 1 + 2 + 1 + 1 = 10$ calls, hence at least ten user-to-kernel crossings (and ten matching kernel-to-user returns). A larger file repeats the read/write pair, so the count grows with the number of chunks, not with the number of commands.
2. No. `printf` formats each byte into a user-space buffer. The library calls `write` only when the buffer fills or when it is flushed. If the buffer does not fill before exit, the hundred calls may produce far fewer than one hundred `write` calls — possibly a single flush at exit. The output bytes are the same; the number of boundary crossings is not.
3. Use a block or table in memory. The caller places the seven arguments in a block and passes the block's address in one register. The kernel locates the block, copies it into kernel memory, and validates every field — in particular, it must verify that the whole block, and not just its first byte, lies in the caller's address space. Register passing alone would be limited by the number of argument registers; the stack is a viable alternative if the calling convention already places arguments there, but the block is the direct answer to "too many arguments for the registers."
4. The call should fail with `EFAULT`. Dereferencing the pointer would let user code read or overwrite kernel data, which is exactly what the privilege boundary exists to prevent. The kernel checks that the range lies in the user address space before copying; if it does not, the request is rejected without touching the address.
5. In 4 KB chunks, 4 MB = 4096 KB, so the program issues $4096/4 = 1024$ `read` calls and 1024 `write` calls: 2048 system calls, hence 2048 mode-switch round trips. With one large `read` and one large `write`, it issues 2 system calls and 2 round trips. The chunked version pays the fixed cost of crossing the boundary 1024 times more often; the large-transfer version pays it twice, but must hold 4 MB in memory at once. The fixed cost of the crossing dominates the chunked case; the buffer size dominates the large-transfer case.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 8th ed., John Wiley and Sons — Ch. 2 §2.3 (system calls, pp. 55–58) and §2.4 (types of system calls, pp. 58–66). *(Page-cited; index built 2026-10-14.)*

[2] W. Stallings, *Operating Systems: Internals and Design Principles*, 6th ed., Prentice Hall — Ch. 2, Ch. 3 (process description and control), and Ch. 4 §4.3. *(Page-cited; index built 2026-10-14.)*

[3] D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed., O'Reilly Media, 2006 — Ch. 10 (system calls, pp. 398–419), for the concrete x86/Linux entry and exit path. *(Page-cited; index built 2026-10-14.)*

Concepts That Click